

Project Summary: AgentOS – An Operating System-Inspired Runtime for Autonomous AI Agents

High-Level Vision

AgentOS is not a replacement for Linux or Windows. Instead, it is an operating system-inspired control plane/runtime that runs on top of Linux and manages AI agents the way an operating system manages processes.

The central idea is that AI agents become first-class system objects.

Today, most agent frameworks treat agents as Python objects inside an application. AgentOS instead treats them as independent processes managed by a central runtime.

Applications should describe what they want accomplished, while AgentOS decides how agents execute, communicate, wait, recover, and use resources.

Core Philosophy

Traditional operating systems abstract hardware.

AgentOS abstracts intelligence.

Linux manages:

- CPU
- Memory
- Files
- Networking
- Processes

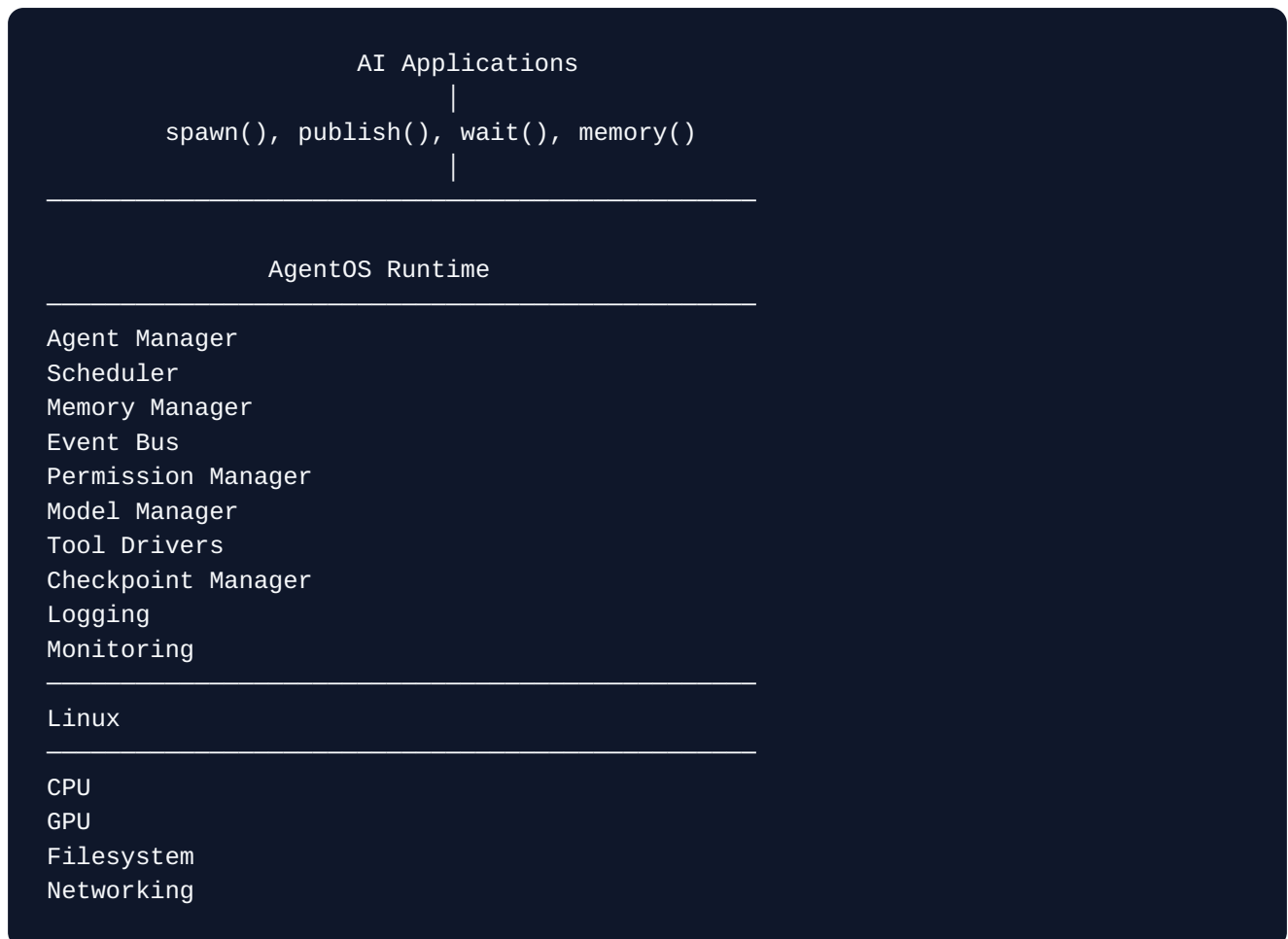
AgentOS manages:

- Agents
- LLMs
- Context
- Memory
- Tool execution
- Human approvals
- Agent communication
- Cost

- Checkpoints
- Dependencies

Linux remains responsible for hardware. AgentOS becomes responsible for intelligent computation.

Overall Architecture



Agent Process Abstraction

Every AI agent is treated as a process.

Instead of

```
research_agent.run()
```

developers create

```
kernel.spawn(ResearchAgent())
```

Every running agent has metadata similar to a Linux process.

Example:

- PID: 17

- Name: Research Agent
- Parent: Planner
- Children: 5
- Status: Running
- Priority: High
- Model: GPT-5
- Permissions: Browser, Python
- Memory: 512 KB
- Checkpoint: #31
- Waiting On: None

The runtime owns every agent. Applications no longer directly orchestrate them.

Agent Lifecycle

Agents transition between states. Possible states include:

- Ready
- Running
- Waiting
- Sleeping
- Blocked
- Finished
- Failed
- Checkpointing
- Suspended

Example:

Planner
↓
Creates Research Agent
↓
Running
↓
Waiting for Search
↓
Running
↓
Waiting for Human Approval
↓

Running

↓

Finished

Scheduler

The scheduler is responsible for deciding which agent executes next. Instead of applications manually calling agents in sequence, developers simply submit work.

Example:

```
kernel.spawn(ProjectPlanner())
```

The scheduler decides:

- execution order
- priorities
- dependency satisfaction
- retries
- starvation prevention
- fairness
- concurrency

Scheduling is based on agent state rather than CPU instructions. Possible scheduling policies:

- FIFO
- Priority scheduling
- Cost-aware scheduling
- Deadline scheduling
- Dependency-aware scheduling
- Round-robin

Future ideas: GPU-aware scheduling, Token-budget scheduling, Energy-aware scheduling

Event Bus

Agents never directly invoke other agents. Instead they publish events.

Example:

Research finishes

↓

Publishes ResearchCompleted

The runtime determines who subscribed.

ResearchCompleted

↓

Wake CodeAgent

↓

Wake DocumentationAgent

↓

Notify Planner

Supported event types:

- AgentFinished
- AgentFailed
- ToolCompleted
- HumanApproved
- MemoryUpdated
- ModelFinished
- TimerExpired
- FileCreated

This produces loosely coupled agent systems.

Dependency Graph

One of the most important features. Instead of static workflows, AgentOS maintains a dependency graph.

Example:

Research depends on: Market Search, Legal Review, Human Approval

Once every dependency completes, the scheduler automatically wakes the waiting process.

Dependencies can include: other agents, tools, timers, events, human approvals, external APIs

Human Approval as a First-Class System Object

Instead of confidence thresholds, agents explicitly declare approval requirements.

Example:

- Need Approval
- Role: Senior Engineer
- Reason: Production deployment

The runtime blocks the process. Status becomes: BLOCKED. Waiting On: Senior Engineer

After approval, the scheduler resumes execution. This treats humans as system resources rather than tools.

Memory Manager

Instead of every framework implementing memory differently, AgentOS provides memory primitives.

Types of memory:

- Working Memory: current reasoning
- Scratchpad: temporary notes
- Shared Memory: accessible by multiple agents
- Long-Term Memory: persistent knowledge
- Semantic Memory: vector storage
- Episodic Memory: execution history

Memory API:

- `memory.store()`
- `memory.retrieve()`
- `memory.share()`
- `memory.delete()`

Implementation could use: SQLite, Redis, Vector database, Filesystem, RAM cache. Applications remain unaware of the backend.

Tool Driver System

Tools behave like operating system device drivers. Instead of importing GitHub libraries directly, agents request capabilities.

Example:

Need: GitHub

The runtime selects the correct driver. Possible drivers:

- Python
- Browser
- SQL
- GitHub
- Filesystem
- Docker
- Shell

- Email
- Calendar
- Slack
- REST APIs
- Future MCP servers

Driver responsibilities: authentication, permissions, rate limiting, retry logic, logging, caching, error handling

Permission System

Every process receives capabilities.

Example:

- Research Agent: ✓ Browser, ✓ Search, ✗ SQL, ✗ Shell
- Coder: ✓ Python, ✓ GitHub, ✓ Docker
- Finance: ✓ SQL, ✓ Email, ✗ Browser

Kernel validates permissions before every tool execution. Possible future additions: role inheritance, temporary permissions, approval-based permissions

Model Manager

Agents never choose models directly. Instead they request capabilities.

Example:

Need: Large reasoning model

The runtime selects: GPT-5, Claude, Gemini, Local model, Reasoning model, Code model, Vision model

Selection criteria: cost, latency, context size, availability, GPU memory, quality

Future: automatic model benchmarking, dynamic model switching

Checkpoint Manager

Running agents periodically save state. Checkpoint includes:

- reasoning
- memory
- tool outputs
- current task

- dependency graph
- event subscriptions

If interrupted, the runtime restores from the last checkpoint. Applications no longer restart entire workflows.

Agent Tree

Agents can spawn child agents.

```
Example:
Planner
├─ Research A
├─ Research B
├─ Research C
└─ Documentation
```

Parent agents: create children, wait for children, terminate children, aggregate results

Shared Runtime

Unlike normal Python libraries, AgentOS exists independently of applications. Multiple applications connect to the same runtime.

Application A, Application B, Application C

↓

AgentOS

The runtime maintains global knowledge of: all agents, all models, all tool usage, all GPU utilization, all costs, all events. This enables global optimization.

Monitoring

Provide both CLI and Web UI.

CLI: agent ps, agent top, agent logs, agent kill, agent wait, agent events

Dashboard: Running Agents, Waiting Agents, Blocked Agents, Dependency Graph, Memory Usage, GPU Usage, Model Usage, Tool Usage, Event Timeline, Cost, Latency

Logging

Every event is recorded. **Example:**

- 12:30 - Planner Spawned

- 12:31 - Research Started
- 12:33 - Browser Search
- 12:34 - Checkpoint Saved
- 12:35 - Human Approval Requested
- 12:40 - Approved
- 12:41 - Execution Resumed

APIs

Example API:

- `kernel.spawn()`
- `kernel.kill()`
- `kernel.pause()`
- `kernel.resume()`
- `kernel.wait()`
- `kernel.publish()`
- `kernel.subscribe()`
- `kernel.memory.store()`
- `kernel.memory.retrieve()`
- `kernel.request_tool()`
- `kernel.request_model()`
- `kernel.checkpoint()`
- `kernel.logs()`

Suggested Tech Stack

- Language: Python
- Backend: FastAPI
- Database: SQLite initially, Later PostgreSQL, Optional Redis
- Vector Database: FAISS or Chroma
- Async Runtime: asyncio
- Visualization: React, D3.js or Cytoscape
- LLM Routing: LiteLLM
- Monitoring: WebSockets
- Docker deployment

Folder Structure

```
agentos/  
  kernel/  
    scheduler.py  
    process.py  
    memory.py  
    permissions.py  
    events.py  
    models.py  
    checkpoints.py  
    tools.py  
  runtime/  
    daemon.py  
    executor.py  
  agents/  
    planner.py  
    research.py  
    coder.py  
    reviewer.py  
  drivers/  
    browser.py  
    python.py  
    github.py  
    sql.py  
    shell.py  
  api/  
    FastAPI  
  dashboard/  
    React UI  
  examples/  
    software_company/  
    research_assistant/  
    customer_support/
```

Long-Term Vision

AgentOS should become a standard execution platform for autonomous AI agents, analogous to how Linux is the standard execution platform for traditional software.

Its purpose is not to replace Linux, but to provide a consistent, operating system-inspired abstraction for intelligent computation.

Instead of every AI application reinventing orchestration, memory, permissions, scheduling, checkpoints, and tool integration, developers build against AgentOS's APIs.

The runtime becomes the central authority responsible for agent lifecycle, communication, resource allocation, dependency resolution, and recovery.

Research Contributions / Novel Ideas

The project's novelty should come from OS-inspired abstractions for AI, not from wrapping existing agent frameworks. Potential contributions include:

- Agents as first-class processes with lifecycle states, PIDs, parent-child relationships, and managed execution.
- Dependency-aware scheduling, where agents are resumed automatically when required agents, tools, timers, or approvals complete, rather than following predefined workflows.
- Human approvals as kernel objects, allowing humans to participate in dependency graphs instead of acting as ad hoc fallbacks.
- Capability-based tool access, where agents request capabilities and the runtime enforces permissions independently of the application.
- Centralized model routing and resource management, enabling cost, latency, and hardware-aware optimization across multiple applications.
- A shared runtime that can coordinate many independent AI applications, providing global visibility into agents, memory, models, events, and costs.
- The overarching goal is to shift AI development from application-managed orchestration to runtime-managed intelligent execution, much as operating systems shifted software development from manually managing hardware to programming against standardized system abstractions.